11.23.4. Product Attestation Authority and Intermediate Certificate Schema

This schema allows approved Product Attestation Authorities Certificates to be uploaded and made available via DCL queries. Information in this schema is populated by the respective vendors who have approved Product Attestation Authorities Certificates. It allows vendors to upload Product Attestation Intermediate certificate under following conditions:

- A corresponding Product Attestation Authorities Certificates exists in DCL that has signed the Product Attestation Intermediate certificate, and
- Both Product Attestation Authorities Certificates and Product Attestation Intermediate certificate are associated with the same Vendor ID.

Note that Matter Operational Root CA Certificates always appear in Matter TLV Certificate format when they appear in cluster data fields. Careful conversions from PEM to Matter TLV Certificate format SHOULD be done whenever a comparison is made.

Name	Type	Constraint	Conformance	Mutable
PEMCert	string	all	M	No
SerialNumber	octstr	20	M	No
Issuer	string	all	O	No
AuthorityKeyID	string	all	O	No
RootSubject	string	all	O	No
RootSubjectKeyID	string	all	O	No
isRoot	boolean	all	M	No
Owner	string	all	M	No
Subject	string	all	M	No
SubjectKeyID	string	all	M	No
GrantApprovals	vendor-id	all	M	No
GrantRejects	vendor-id	all	M	No
VID	vendor-id	all	M	No
SchemaVersion	uint16	all	M	Yes

11.23.4.1. PEMCert

This field uniquely identifies a certificate and SHALL contain the body of a certificate that has been added in the DCL. It SHALL be encoded in PEM format. The certificate SHALL respect the format constraints provided for that certificate type.

11.23.4.2. SerialNumber

The field SHALL be used to identify the serial number field in the Matter certificate structure. A Matter certificate follows the same limitation on admissible serial numbers as in [RFC 5280], i.e., that implementations SHALL admit serial numbers up to 20 octets in length, and certificate authori-

ties SHALL NOT use serial numbers longer than 20 octets in length.

11.23.4.3. Issuer

The field SHALL be used to identify the Certificate Authority that issues the certificate. For a PAA Certificate, this field is OPTIONAL because Issuer and Subject are the same.

11.23.4.4. AuthorityKeyID

The authority key identifier extension provides a means of identifying the public key corresponding to the private key used to sign a Matter certificate. This is OPTIONAL for PAA Certificates.

11.23.4.5. RootSubject

This field SHALL contain the PAA certificate's `Subject` field, as defined in PAA in PAA Certificate. This is OPTIONAL for PAA Certificates. This is encoded as defined in Section 6.1, "Certificate Common Conventions".

11.23.4.6. RootSubjectKeyID

This field SHALL uniquely identify the PAA certificate's `Subject Key Identifier` mandatory extension. It is defined in PAA Certificate and Operational Root CA Certificates (RCAC). This is OPTIONAL for PAA Certificates. This is encoded as defined in Section 6.1, "Certificate Common Conventions".

11.23.4.7. Owner

This field uniquely identifies the DCL key that was used to register the certificate in DCL, pursuant to DCL policies.

11.23.4.8. isRoot

This field SHALL signify whether the associated certificate is PAA Certificate.

11.23.4.9. Subject

This field SHALL contain the certificate's `Subject` field. This is OPTIONAL for PAA Certificates. This is encoded as defined in Section 6.1, "Certificate Common Conventions".

11.23.4.10. SubjectKeyID

This field SHALL uniquely identify the PAA certificate's `Subject Key Identifier` mandatory extension. This is encoded as defined in Section 6.1.2, "Key Identifier Extension Constraints".

11.23.4.11. GrantApprovals

This field SHALL contain list of DCL Keys that approved the PAA Certificate admission into DCL. This field SHALL be set only for a PAA Certificate.

11.23.4.12. GrantRejects

This field SHALL contain list of DCL Keys that rejected the PAA Certificate admission into DCL. This

Any client that discovers a Anchor Node with DNS-SD and connects to the Node via CASE SHALL check if the Anchor CAT or the Anchor/Datastore CAT is present in the NOC of the Node and the NOC chains up to the Anchor ICAC.

If the Node also has the role of a Datastore then the Anchor/Datastore CAT SHALL be used instead.

12.2.4.3. Datastore CAT

A NOC containing the Datastore CAT SHALL be issued by any Joint Fabric Administrator.

Any client that discovers a Datastore Node with DNS-SD and connects to the Node via CASE SHALL check if the Datastore CAT or the Anchor/Datastore CAT is present in the NOC of the NODE as part of the CASE handshake. This SHALL be required in order to verify that the Node is authorized to act as a Datastore in the Joint Fabric.

If the Node is also a Joint Fabric Anchor then the Anchor/Datastore CAT SHALL be used instead.

12.2.4.4. Anchor/Datastore CAT

A NOC containing the Anchor/Datastore CAT SHALL be issued only by the Anchor ICAC.

Any client that discovers an Anchor/Datastore Node with DNS-SD and connects to the Node via CASE SHALL check the Anchor/Datastore CAT is present in the NOC of the Node as part of the CASE handshake. This SHALL be required in order to verify that the Node is authorized to act as an Anchor and Datastore in the Joint Fabric.

12.2.5. Joint Commissioning Method (JCM)

NOTE	Support for Joint Commissioning Method is provisional.

This method SHALL be implemented for Commissioners and Administrators that support Joint Fabric. Given a pre-existing Matter fabric Fabric A managed by an Ecosystem A with its own administrator nodes and commissioner nodes, a Joint Fabric can be created with another Ecosystem B such that all devices of Ecosystem A and Ecosystem B can communicate using a single operational root of trust and devices can be added to the new joint fabric by either Ecosystem.

The joint operation ensures that ICA's from both Fabric A and Fabric B are signed by the Anchor CA (the Root CA of the fabric selected to become the Anchor Fabric), establishing a common trust between all devices of the original Fabric A and Fabric B and all newly added devices to the "Joint Fabric".

The Joint Fabric contains multiple ecosystems whose ICACs are cross-signed by the Anchor CA. Therefore, all ecosystems participating in the Joint Fabric SHALL use an ICAC to sign NOCs. Signing NOCs with a Root CA different than the Anchor CA is not permitted in the Joint Fabric.

The following steps describe JCM. In the figure below, Ecosystem A with Fabric A acts as the Anchor Fabric and Ecosystem B with Fabric B as joining fabric to create a Joint Fabric between the two Ecosystems.

1. Commission an Administrator from the Ecosystem B (Eco B Admin) using either ECM or BCM

triggered by one Commissioner device from Ecosystem A (Eco A Commissioner). When Ecosystem B Administrator advertises its presence over DNS-SD (as a result of Open Commissioning Window) it SHALL also advertise the correct JF TXT key.

2. Once ECM or BCM is over, Ecosystem B Administrator SHALL check that the `NOC` belonging to Ecosystem A Administrator (`responder NOC` field of the sigma-2-tbsdata) contains either an Anchor CAT or an Anchor/Datastore CAT.
3. User Consent must be obtained on Ecosystem B before proceeding with the next steps.
4. Ecosystem B Administrator SHALL use ec-pub-key, pub-key-algo and ec-curve-id of Eco A RCAC together with Ecosystem A VendorID as input parameters for RCAC based Vendor ID validation.
5. Ecosystem B Administrator SHALL check that the returned certificate is a RCAC (Basic Constraint extension SHALL be marked `critical` and have the `cA` field set to TRUE).
6. Ecosystem B Administrator requests an Administrator from Ecosystem A to sign its ICA by invoking the ICACSR Request command of the Joint Fabric PKI Cluster, as described in ICA Cross Signing.
7. Ecosystem A Administrator performs the steps in ICA Cross Signing on the request from the Ecosystem B Administrator.
8. Cross-signed ICAC is returned to Ecosystem B Administrator using ICACSR Response command.
9. Ecosystem B Administrator SHALL use following commands from Node Operational Credentials cluster to update each node in the Fabric B to join the Joint Fabric:
    a. Update TrustedRootCertificates by using AddTrustedRootCertificate
    b. Update Node Operational Credentials using AddNOC. Subject DN of the issued NOCs encodes a `matter-fabric-id` attribute whose value SHALL be identical with the value of the `matter-fabric-id` attribute from the cross-signed ICAC.
    c. Remove the old Trusted Root CA Certificate and the associated fabric using the RemoveFabric command

Upon successful completion of JCM, all nodes from Fabric B will be part of the Joint Fabric anchored by the Ecosystem A.

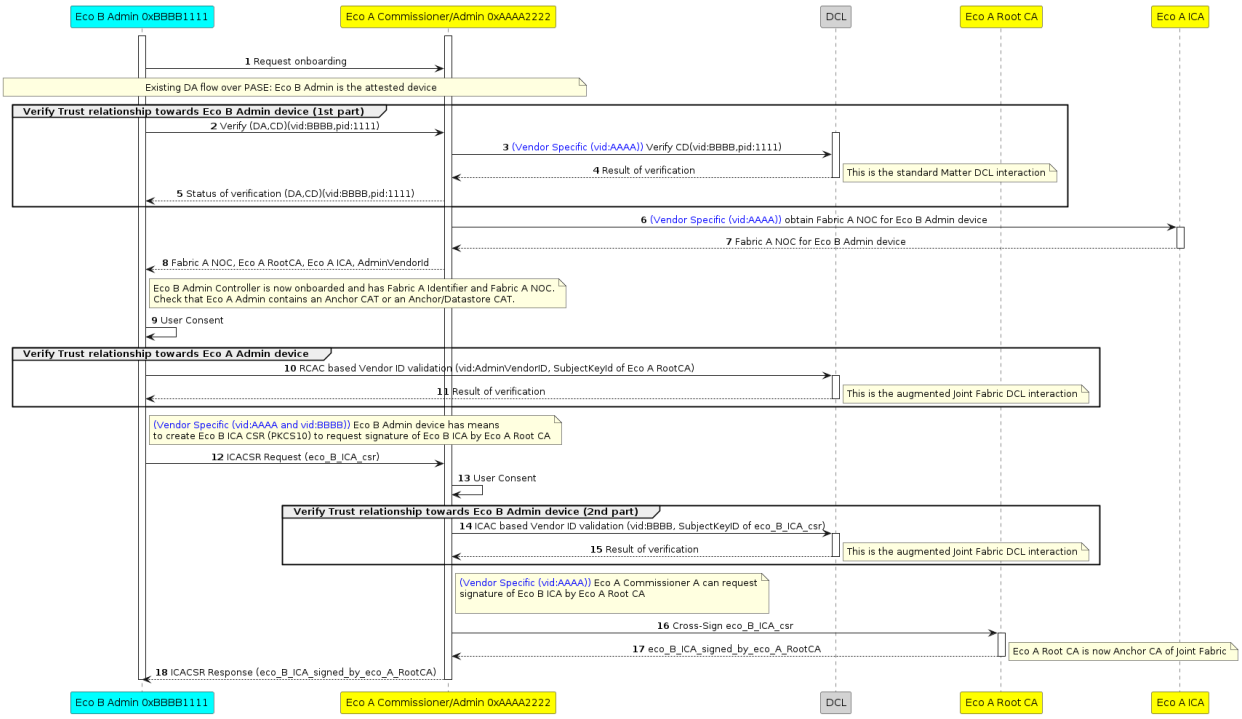

Figure 102. Joint Commissioning Steps

12.2.5.1. Scope of User Consent

Before commissioning a Joint Fabric Administrator, the user SHALL be asked for consent to enable Joint Fabric functionality between ecosystems. Each ecosystem joining the Joint Fabric SHALL independently ask the user for consent.

12.2.5.2. Discovery

The user SHALL be able to enable JCM through an appropriate interface of the devices on that ecosystem. For example, a mobile application, a web configuration, or an on-device interface.

12.2.5.3. Vendor ID Validation

The Joint Fabric requires that the fabrics participating in a Joint Fabric perform mutual Vendor ID validation so that they can be sure of the authenticity of the Vendor ID being asserted. Vendor ID validation SHALL be achieved by using the VendorID Validation Procedure.

12.2.5.4. ICA Cross Signing

As described in the Joint Commissioning Method, an Ecosystem Administrator, different than the Joint Fabric Anchor Administrator, SHALL have the ability to issue NOCs chaining up to the Anchor CA. To obtain this ability it SHALL receive an ICAC issued by the Joint Fabric Anchor Administrator. Cross-signing means that the SubjectPublicKey of the ICAC chaining up to the Anchor CA is identical to the SubjectPublicKey of the pre-installed ICAC (chaining up to that particular Ecosystem Administrator RCAC).

The Ecosystem Administrator that needs to obtain a cross-signed ICAC SHALL create a Certificate Signing Request (ICA CSR) as described in PKCS #10:

   a. If the output of the validation is an empty list, error SHALL be FailedDCLVendorIdValidation
   b. Check that the DCL returned certificate is an ICAC (Basic Constraints extension SHALL be marked `critical` and have the `cA` field set to FALSE).
      i. If not an ICAC, error status SHALL be NotAnIcac.
6. Upon success, ICA's certificate SHALL be signed by the RCAC of the Joint Fabric Anchor Administrator.
   a. If signing fails, error status SHALL be IcaCsrSigningFailed.
7. ICAC SHALL be returned in a PEM format inside the ICAC field of the ICACSRResponse command.

If any of the above validation checks fail, the server SHALL immediately respond to the client with an ICACSRResponse command. The StatusCode SHALL be set to the error status value specified in the above validation checks.

12.2.6. Anchor Administrator Selection

The Joint Fabric design allows devices to communicate with one another regardless of which commissioner was used to commission devices. Similarly, devices participating in Joint Fabric can be removed without impacting the remaining devices. If all devices commissioned using a particular commissioner are deleted, the devices remaining in the Joint Fabric will typically continue to function.

User SHALL provide consent when an Anchor Administrator is removed and a new Anchor CA is chosen. In the example below, the Anchor Administrator is removed from the Joint Fabric and the user has selected a candidate Administrator to be promoted to Anchor Administrator. For clarity, Joint Fabric Anchor Administrator `A` is being replaced with the candidate Joint Fabric Administrator `B`.

User consent SHALL be mutual:

- on Administrator `A` side, User provides consent that transfer of the Anchor role to Administrator `B` is allowed.
- on Administrator `B` side, User provides consent that receipt of the Anchor role from Administrator `A` is allowed.

Flow for Anchor Transfer is as follows:

1. Administrator B SHALL:
   a. send TransferAnchorRequest to Administrator A to set itself as Joint Fabric Anchor Administrator.
   b. obtain user consent prior to sending the TransferAnchorRequest command.
2. Administrator A SHALL:
   a. check that the NOC used by Administrator B during the CASE session contains an Administrator CAT.
   b. check that user provided consent that allows the transfer of the Anchor role to a different

Administrator. If not, then TransferAnchorResponse command with StatusCode set to TransferAnchorStatusNoUserConsent SHALL be sent and the procedure stopped here.

c. check all the Datastore entries of type DatastoreStatusEntry. If any of these entries has a value that equals to `Pending` or to `DeletePending` then TransferAnchorResponse command with StatusCode set to TransferAnchorStatusDatastoreBusy SHALL be sent and the procedure stopped here.

d. put Joint Fabric Datastore in read only state by setting the Datastore StatusEntry to `DeletePending`.

e. stop DNS SD advertising of the `Administrator`, `Anchor` and `Datastore` capability inside the JF TXT key.

f. set BusyAnchorTransfer error status for the ICACSRResponse in case an ICA Cross Signing is in progress.

3. All other Joint Fabric Administrators SHALL:

a. stop commissioning of any new devices into the Joint Fabric once it detects that Datastore StatusEntry equals `DeletePending`.

4. Administrator B SHALL:

a. copy (through attribute read, BDX) Joint Fabric Datastore from Administrator A.

b. set the Datastore StatusEntry to `Committed`.

c. set the value of the Datastore AnchorNodeId attribute to the value of its Node ID.

d. increase Administrator CAT version.

e. use following commands from Node Operational Credentials cluster on the other Joint Fabric Administrators:

i. update TrustedRootCertificates by using AddTrustedRootCertificate.

ii. update Node Operational Credentials using AddNOC. `NOCs` SHALL contain an updated version of the Administrator CAT.

f. issue itself a new NOC with the updated version of the Anchor/Datastore CAT.

g. set itself as the new Datastore provider and Anchor Administrator by advertising the `Administrator`, `Anchor` and `Datastore` capabilities inside the JF TXT key.

h. send TransferAnchorComplete to Administrator A to announce that transition to Anchor Administrator is complete.

5. All other Joint Fabric Administrators SHALL:

a. subscribe to the new Datastore provider (Administrator B) having discovered the new `Datastore` capability via Service Discovery of the JF TXT key.

b. check that the NOC used by the new Datastore provider (Administrator B) during the first CASE session contains an Anchor/Datastore CAT and that the NOC chains up to the Anchor ICAC.

c. request ICA Cross Signing from the new Joint Fabric Anchor Administrator discovered via Service Discovery (`Administrator` and `Anchor` flags are set in JF TXT key).

d. remove devices from the fabric governed by the old `Anchor CA` using the RemoveFabric com-

resolve to a maintained web page. The syntax of this field SHALL follow the syntax as specified in RFC 1738 and SHALL use the `https` scheme. The maximum length of this field is 256 ASCII characters.

11.23.6.18. LsfUrl

This field (when provided) SHALL identify a link to the Localized String File of this product. This field SHALL NOT have a localized string identifier. During the lifetime of the product, the specified URL SHOULD resolve to a maintained Localized String File. The syntax of this field SHALL follow the syntax as specified in RFC 1738 and SHALL use the `https` scheme. The maximum length of this field is 256 ASCII characters.

11.23.6.19. LsfRevision

LsfRevision is a monotonically increasing positive integer indicating the latest available version of Localized String File. Any client can use this field to check whether it has the latest version of the Localized String File cached. When the first version of the Localized String File is published, the value of this field SHOULD be 1. When a new revision of the Localized String File is published, this value SHALL monotonically increase by 1. When a client of this information finds this to be greater than its currently stored LSF revision number, it SHOULD download the latest version of the LSF from the LsfUrl, and update its local value of this field.

This field SHALL be provided if and only if when LsfUrl is provided.

11.23.6.20. SchemaVersion

The SchemaVersion field value history for this schema is provided below:

Version	Description
0	Initial Release

11.23.6.21. EnhancedSetupFlowOptions

This field SHALL identify the configuration options for the Enhanced Setup Flow. This field is a bitmap with values defined in Enhanced Setup Flow Options Table.

11.23.6.22. EnhancedSetupFlowTCUrl

This field (when provided) SHALL identify a link to the Enhanced Setup Flow Terms and Condition File for this product. During the lifetime of the product, the specified URL SHOULD resolve to a maintained Terms and Conditions File. The syntax of this field SHALL follow the syntax as specified in RFC 1738. The maximum length of this field is 256 ASCII characters. All URLs SHALL use the `https` scheme. This field SHALL be present if and only if the EnhancedSetupFlowOptions field has bit 0 set.

11.23.6.23. EnhancedSetupFlowTCRevision

This field (when provided) is an increasing positive integer indicating the latest available version of the Enhanced Setup Flow Terms and Conditions file. When a new revision of the File is published,

this value SHALL increase (and SHOULD increase by 1). This field SHALL be present if and only if the EnhancedSetupFlowOptions field has bit 0 set.

11.23.6.24. EnhancedSetupFlowTCDigest

This field (when provided) SHALL contain the digest of the entire contents of the associated file downloaded from the EnhancedSetupFlowTCUrl field, encoded in base64 string representation and SHALL be used to ensure the contents of the downloaded file are authentic. The digest SHALL have been computed using the SHA-256 digest algorithm. This field SHALL be present if and only if the EnhancedSetupFlowOptions field has bit 0 set.

11.23.6.25. EnhancedSetupFlowTCFileSize

This field (when provided) SHALL indicate the total size of the Enhanced Setup Flow Terms and Conditions file in bytes, and SHALL be used to ensure the downloaded file size is within the bounds of EnhancedSetupFlowTCFileSize. This field SHALL be provided if and only if the EnhancedSetupFlowTCUrl field is present.

11.23.6.26. EnhancedSetupFlowMaintenanceUrl

This field (when provided) SHALL identify a link to a vendor-specific URL which SHALL provide a manufacturer specific means to resolve any functionality limitations indicated by the TERMS_AND_CONDITIONS_CHANGED status code. This field SHALL be present if the EnhancedSetupFlowOptions field has bit 0 set. During the lifetime of the product, the specified URL SHOULD resolve to a maintained web page. The syntax of this field SHALL follow the syntax as specified in RFC 1738. The maximum length of this field is 256 ASCII characters. All URLs SHALL use the `https` scheme.

11.23.7. DeviceSoftwareVersionModel Schema

A unique combination of the VendorID, ProductID and SoftwareVersion is used to identify a DeviceSoftwareVersionModel. The record schema available to vendors to provide information specific to a particular software version for a given product is presented below.

Name - (Matching Basic Information Cluster Field)	Type	Constraint	Conformance	Mutable
VendorID	vendor-id	all	M	No
ProductID	uint16	all	M	No
SoftwareVersion	uint32	all	M	No
SoftwareVersionString	string	1 to 64	M	No
CDVersionNumber	uint16	all	M	No
FirmwareInformation	string	max 512	O	No

Range	Type
0xFFFF_FFFF_0000_0000 to 0xFFFF_FFFF_FFFE_FFFF	Reserved for future use
0xFFFF_FFFE_xxxx_xxxx	Temporary Local Node ID
0xFFFF_FFFD_xxxx_xxxx	CASE Authenticated Tag
0xFFFF_FFFC_xxxx_xxxx to 0xFFFF_FFFC_FFFF_FFFF	Reserved for future use
0xFFFF_FFFB_xxxx_xxxx	PAKE key identifiers
0xFFFF_FFF0_0000_0000 to 0xFFFF_FFFA_FFFF_FFFF	Reserved for future use
0x0000_0000_0000_0001 to 0xFFFF_FFEF_FFFF_FFFF	Operational Node ID
0x0000_0000_0000_0000	Unspecified Node ID

Node IDs are used for core messaging, within the internal APIs, within the data model, and to resolve the operational IPv6 addresses of Nodes (see Section 4.3.2, “Operational Discovery”).

The span of Node IDs from 0xFFFF_FFF0_0000_0000 to 0xFFFF_FFFF_FFFF_FFFF, as well as the value 0x0000_0000_0000_0000 are both reserved for special uses.

2.5.5.1. Operational Node ID

An Operational Node ID is a 64-bit number that uniquely identifies an individual Node on a Fabric. All messages must have an Operational Node ID as the source address. All unicast messages must have an Operational Node ID as the destination address.

While source or destination address MAY be elided from a message, it MUST remain unambiguously derivable from the Session ID.

2.5.5.2. Group Node ID

A Group Node ID is a 64-bit Node ID that contains a particular Group ID in the lower half of the Node ID.

2.5.5.3. Temporary Local Node ID

A Temporary Local Node ID is a 64-bit Node ID that contains an implementation-dependent value in its lower 32 bits. This allows implementations to keep track of connections or transport-layer links and similar housekeeping internal usage purposes in contexts where an Operational Node ID is unavailable.

2.5.5.4. PAKE key identifiers

This subrange of Node ID is used to assign an access control subject to a particular PAKE key as specified in Section 6.6.2.1.1, “PASE and Group Subjects”. An example usage would be to create an ACL entry to provide administrative access to any commissioner communicating via a PASE session

Chapter 12. Multiple Fabrics

12.1. Introduction

The Multiple Fabric feature allows a Node to be commissioned to multiple separately-administered Fabrics. With this feature a current Administrator can (with user consent) allow the Commissioner for another fabric to commission that Node within its Fabric. The new Commissioner MUST have their own Node Operational Certificate (NOC) issued by its Trusted Root Certificate Authority (TRCA). Once commissioning is completed and the Node is properly configured, Administrators on the newly joined Fabric have access to the Node and can perform all administrative tasks.

A Fabric is anchored by its Trusted Root Certificate Authority (TRCA). A TRCA MAY delegate to one or more Intermediate Certificate Authorities (ICA) which issue NOCs. Multiple vendors or companies can use the same CA hierarchy in which case they will be governed under the same Trusted Root Certificate Authority.

12.2. Joint Fabric

12.2.1. Introduction

When multiple vendors or companies use the same CA hierarchy, governed under the same Trusted Root Certificate Authority, they create a Joint Fabric using the Joint Commissioning Method flow. This flow enables a set of one or more companies to use a single fabric anchored by the Trusted Root Certificate Authority (TRCA). In the context of JCM, the fabric that is anchored by this TRCA is known as the Anchor Fabric.

NOTE	Support for Joint Fabric is provisional.

12.2.2. Node ID Generation

Any newly-allocated Node ID SHALL:

- be greater than 0x0000_0000_0000_0000, but less than 0xFFFF_FFEF_FFFF_FFFF, representing a value within the Operational NodeID range (see Table 4, “Node Identifier Allocations”);
- be checked to ensure its uniqueness in the NodeList attribute of the Joint Fabric Datastore

The Node ID SHALL be regenerated if these constraints are not met.

It is RECOMMENDED to use random allocation within the valid range to avoid having to regenerate the Node ID.

12.2.3. Anchor ICAC requirements

Anchor ICAC SHALL be the ICAC corresponding to the Anchor Administrator. Anchor ICAC SHALL contain the reserved org-unit-name attribute from the Table 64, “Standard DN Object Identifiers” with value jf-anchor-icac in its Subject DN. The Anchor CA SHALL NOT place the reserved org-unit-name attribute jf-anchor-icac value into any Node that is not the Anchor Administrator. The

